Floci AWS Documentation

1. Docker and Floci inspect

```
docker ps -a
docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}
\t{{.Ports}}'
docker logs --tail 200 floci 2>&1 | grep -iE 'error|warn|exception|fail|
pull'
df -h / ; free -h
```

```
docker inspect floci -f '{{json .HostConfig.PortBindings}}'
docker inspect floci -f '{{range .Config.Env}}{{println .}}{{end}}' |
grep -i floci
docker inspect floci -f '{{range .Mounts}}{{.Source}} -> {{.Destination}}
{{println}}{{end}}'
```

```
API and CLI
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:4566
env | grep -iE 'AWS_|proxy'
aws configure list
```

```
mkdir -p ~/triage && cd ~/triage
docker ps -a > containers-before.txt
docker inspect floci > floci-before.json
cp <their-folder>/docker-compose.yml compose-before.yml 2>/dev/null
```

```
Docker Host
systemctl is-active docker          # active
docker info | grep -iE 'Server Version|Storage Driver|Total Memory'
df -h /var/lib/docker; docker system df
free -h; dmesg 2>/dev/null | grep -i -E 'oom|killed process' | tail
```

```
Floci Container
docker ps -a --filter name=floci --format '{{.Names}} {{.Status}}'
docker logs --tail 100 floci
docker inspect floci -f 'restarts={{.RestartCount}}
exit={{.State.ExitCode}} oom={{.State.OOMKilled}}'
```

```
Floci API and CLI
curl -v http://localhost:4566 2>&1 | head -5
export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test
AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=<their region>
export NO_PROXY=localhost,127.0.0.1 no_proxy=localhost,127.0.0.1
aws sts get-caller-identity
```

```
Resources
aws ec2 describe-instances --query 'Reservations[].Instances[].
[InstanceId,State.Name,PrivateIpAddress,ImageId]' --output table
aws rds describe-db-instances --query 'DBInstances[].
[DBInstanceIdentifier,Engine,DBInstanceStatus,Endpoint.Port]' --output
table
aws ec2 describe-vpcs --output table
# not there? try other regions
for r in us-east-1 us-west-2 eu-west-1 ap-south-1; do echo $r; aws ec2
describe-instances --region $r --query 'length(Reservations)'; done
```

```
Spawned containers
docker ps -a --filter name=floci-aws --format 'table {{.Names}}
\t{{.Status}}\t{{.Ports}}'
docker logs --tail 50 <floci-aws-ec2-...|floci-aws-rds-...>
docker exec -it <floci-aws-ec2-...> bash   # look inside the EC2
```

```
Network
docker network ls
docker inspect -f '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}
{{$v.IPAddress}}{{println}}{{end}}' <container>
nc -zv -w 5 <host> <port>                  # from the host
docker exec <ec2> nc -zv -w 5 <host> <port>  # from inside the EC2
sudo ss -ltnp | grep -E ':4566|:7001|:80 |:443 '
sudo ufw status
getent hosts <name>
```

```
Application
docker exec <ec2> sh -c 'ps -eo pid,stat,cmd | grep -v " Z "'
docker exec <ec2> ss -ltnp
docker exec <ec2> tail -50 <app log>
docker exec <ec2> cat <app config>
```

Symptom	Likely cause	Check	Fix
`Could not connect to the endpoint URL: http://localhost:4566/` (seen)	Floci stopped; port not published; proxy env vars	`docker ps -a`; `curl -v localhost:4566`; `env | grep -i proxy`	`docker compose up -d`; publish 4566; `export NO_PROXY=localhost,127.0.0.1`
`NoRegion` (seen)	Region not set in this shell	`aws configure list`	`export AWS_DEFAULT_REGION=<region>`
Resources "disappeared"	Wrong region, or Floci lost its state on restart	List in other regions; `FLOCI_STORAGE_MODE` and data mount	Use the right region; persistent storage + volume
`expected one argument` / `No such container: tail` (seen)	Shell variables empty (new terminal)	`echo $VAR`	Reload or export the variables
`DBInstanceAlreadyExists` then `DBInstanceNotFound` (seen)	Engine image still pulling; create timed out	`docker logs floci | grep -i pull`	`docker pull <image>`, then delete and create again
`Transient image pull failure`	Slow or no internet; registry blocked; proxy	`docker pull <image>` by hand	Pre-pull; fix proxy; load image from a tar
No containers start for EC2, RDS or Lambda	`docker.sock` not mounted, or Floci lacks permission	`docker inspect floci` mounts; logs say permission denied	Mount `/var/run/docker.sock`; `FLOCI_RUN_AS_ROOT=true`
RDS `available` but connection refused	Proxy ports 7001-7099 not published; wrong port	`describe-db-instances` port; `nc -zv localhost 7001`	Publish `7001-7099`; use the reported port
`Login failed for user`	Wrong master password, or engine still starting	RDS container logs	Use the create password; wait 30-60 s
`CREATE DATABASE permission denied` (seen)	Floci master login has fewer rights than real RDS	Error text in app log	Create the DB as `sa` inside the RDS container; grant `db_owner`
App in EC2 cannot reach RDS	Using Floci container IP or wrong host	`echo $AWS_ENDPOINT_URL` inside EC2; `nc -zv`	Host from `AWS_ENDPOINT_URL` (e.g. `localhost.floci.io`) + proxy port
EC2 container exits or runs wrong binaries	Wrong CPU architecture (`ami-`	`docker inspect <ec2> -f '{{.Config.Image}}'`	Use `ami-ubuntu2404-amd64` or `ami-`

Symptom	Likely cause	Check	Fix
	ubuntu2404 alias = arm64)	; uname -m	ubuntu2204
User-data did nothing	No shebang, script error, not base64 when sent raw	docker logs floci | grep -i user-data; /var/lib/user-data.sh	Start with #!/bin/bash; use --user-data file://...
EC2 gets an unexpected IP range	VPC CIDR not private, or clashes with another Docker network or VPN	Floci WARN about CIDR; docker network ls	Use an RFC 1918 range not in use
bind: address already in use on port 80 inside EC2 (seen)	Floci IMDS socat holds 169.254.169.254:80	ss -ltnp inside the EC2	Bind the app to the instance IP (Caddy default_bind)
address already in use on the host	Another process or container on 4566, 7001 or 2200	sudo ss -ltnp | grep <port>	Stop the other process or change the port
Duplicate or zombie processes (seen)	Old processes never cleaned up	ps -eo pid,stat,cmd (Z = zombie)	Kill by PID or docker restart <ec2>
Could not resolve host (seen)	Name missing from /etc/hosts	getent hosts <name>	Add <ip> <name> to /etc/hosts
Exited (137) or app killed	Out of memory, or instance-type memory cap	free -h; docker stats; OOMKilled	Free memory; larger instance type
Container cannot reach the host	Host firewall	sudo ufw status	Allow the Docker bridge
Image 404 / Firefox OpaqueResponseBlocking (seen)	App stores files in one folder, web server serves another	curl -I <image url>; find / -name <file>	One absolute path for both
Security group rules not enforced	Floci stores SGs as metadata by default	Floci docs	Explain; enforcement is opt-in
EFS mount fails	Floci EFS has no NFS data plane	mount error	Explain; use a local volume
Disk full, pulls fail	Old images and volumes	docker system df	Prune unused images (ask first)

EC2

```
aws ec2 describe-instances --query 'Reservations[].Instances[].
[InstanceId,State.Name,ImageId,InstanceType,PrivateIpAddress]' --output
table
docker ps -a --filter name=floci-aws-ec2 --format 'table {{.Names}}
\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
docker exec -it floci-aws-ec2-<id> bash
ssh -i <key.pem> -p 2200 root@localhost        # port from docker ps
```

RDS

aws rds describe-db-instances --query 'DBInstances[].
[DBInstanceIdentifier,Engine,DBInstanceStatus,Endpoint.Address,Endpoint.Port] ' --output table
docker logs floci 2>&1 | grep -iE 'rds|proxy' | tail
docker logs --tail 50 floci-aws-rds-db-
nc -zv localhost # SQL Server login test through the proxy
docker run --rm --network host --entrypoint /opt/mssql-tools18/bin/sqlcmd mcr.microsoft.com/mssql/server:2022-latest -S 127.0.0.1, -U -P '' -C -Q 'SELECT @@VERSION'
psql -h localhost -p -U
mysql -h 127.0.0.1 -P -u -p

VPC

aws ec2 describe-vpcs --output table
aws ec2 describe-subnets --query 'Subnets[].
[SubnetId,VpcId,CidrBlock,AvailabilityZone]' --output table
aws ec2 describe-security-groups --query 'SecurityGroups[].
[GroupId,GroupName,VpcId]' --output table docker network ls;
docker network inspect

S3

aws s3 ls
aws s3 mb s3://test-bucket && echo hi > t.txt && aws s3 cp t.txt s3://test bucket/ && aws s3 ls s3://test-bucket/

SQS / SNS

aws sqs list-queues
aws sqs send-message --queue-url --message-body test
aws sqs receive-message –queue-url>

DynamoDB

aws dynamodb list-tables
aws dynamodb describe-table –table-name
aws dynamodb scan --table-name --max-items 5

Ports

Port	What	Notes
4566	Floci AWS API	All services; set `AWS_ENDPOINT_URL` to it
4577	Floci Azure API	Separate `floci-az` emulator
4500	Floci UI	Separate `floci-ui` container
7001-7099	RDS proxy	Must be published on the Floci container
2200-2299	EC2 SSH	Published by each EC2 container itself
30000-30999	Security group forwards	socat sidecars when SG ports are open to a CIDR
9169	EC2 metadata (IMDS)	Inside instances at `169.254.169.254:80`

Key settings (environment variables on the Floci container)

Setting	Purpose
`FLOCI_STORAGE_MODE`	Persistence of resource state (used `hybrid` with `./data` mounted)
`FLOCI_RUN_AS_ROOT`	`true` when the Docker socket is root-only
`FLOCI_SERVICES_DOCKER_NETWORK`	Docker network for spawned containers
`FLOCI_DOCKER_DOCKER_HOST`	Docker daemon socket, default `unix:///var/run/docker.sock`
`FLOCI_HOSTNAME`	Stable Floci hostname, used by Lambda to call back
`FLOCI_SERVICES_RDS_PROXY_BASE_PORT` / `_MAX_PORT`	RDS proxy port range
`FLOCI_SERVICES_RDS_ENDPOINT_HOST`	Hostname returned as the RDS endpoint
`FLOCI_SERVICES_RDS_MOCK`	`true` = metadata only, no database container
`FLOCI_SERVICES_RDS_DEFAULT_SQL_SERVER_IMAGE` (and `_POSTGRES_`, `_MYSQL_`, `_MARIADB_`)	Engine image per type
`FLOCI_SERVICES_EC2_PUBLISH_SECURITY_GROUP_PORTS`	`false` = no socat forwarders
`FLOCI_SERVICES_EC2_INSTANCE_RESOURCE_LIMITS`	`false` = no CPU/memory caps per instance type
`FLOCI_SERVICES_EC2_VPC_NETWORKS_ENABLED`	`false` = no Docker network per VPC
`FLOCI_NETWORK_SECURITY_GROUP_ENFORCEMENT_ENABLED`	`true` = enforce SG rules with nftables

AMI to image

AMI	Docker image	Note
`ami-ubuntu2204`	`ubuntu:22.04`	What we used
`ami-ubuntu2404-amd64`	`ubuntu:24.04`	x86 hosts
`ami-ubuntu2404`	`ubuntu:24.04` arm64	Wrong arch on x86 PCs
`ami-amazonlinux2023`	`amazonlinux:2023`	Also the default for unknown AMIs
`ami-debian12`, `ami-alpine`	`debian:12`, `alpine`	

Docker

Command	Use
`docker ps -a`	All containers, including stopped ones
`docker logs --tail 100 -f <c>`	Read or follow a container's log
`docker inspect <c>`	Ports, mounts, env, networks, exit code
`docker exec -it <c> bash`	Shell inside a container (EC2)
`docker exec <c> <cmd>`	Run one command inside
`docker cp <src> <c>:<dst>`	Copy files in or out
`docker restart <c>`	Restart one container
`docker stats --no-stream`	CPU and memory per container
`docker network ls` / `inspect <net>`	Networks, subnets, gateways
`docker system df`	Disk used by images and volumes
`docker compose ps` / `up -d` / `logs -f`	Compose project (run in its folder)
`docker pull <image>`	Pre-download an image

AWS CLI against Floci

Command	Use
`export AWS_ENDPOINT_URL=http://localhost:4566`	Point the CLI at Floci
`export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test`	Dummy keys
`export AWS_DEFAULT_REGION=<region>`	Region
`aws sts get-caller-identity`	Is Floci answering?
`aws configure list`	Which endpoint, region and keys are in use
`aws ec2 describe-instances`	EC2 list and state
`aws ec2 start-instances` / `stop-instances --instance-ids <id>`	Start or stop EC2

Command	Use
`aws rds describe-db-instances`	RDS list, status, port
`aws rds start-db-instance` / `stop-db-instance --db-instance-identifier <id>`	Start or stop RDS
`aws ec2 describe-vpcs` / `describe-subnets` / `describe-security-groups`	Network records
`--region <r>` on any command	Look in another region
`--debug` on any command	See the exact request and endpoint

Network

Command	Use
`curl -v <url>`	Full request and response; TLS errors
`curl -sk -o /dev/null -w '%{http_code}\n' <url>`	Just the status code
`curl -skI <url>`	Response headers (content type)
`nc -zv -w 5 <host> <port>`	Is a TCP port open?
`sudo ss -ltnp`	Who listens on which port
`getent hosts <name>`	Does a name resolve, and to what?
`ping -c 2 <ip>`	Basic reachability
`cat /etc/hosts`	Local name overrides
`sudo ufw status`	Host firewall

Linux

Command	Use
`free -h` / `df -h`	Memory / disk
`dmesg | grep -i oom`	Out-of-memory kills
`ps -eo pid,ppid,stat,cmd`	Processes (`Z` = zombie)
`pkill -f <pattern>` / `kill <pid>`	Stop a process
`tail -f <log>`	Follow a log
`grep -iE 'error|fail' <log>`	Find errors
`find / -xdev -name '<file>' 2>/dev/null`	Find a file anywhere
`env | grep -iE 'AWS_|proxy'`	Shell variables that change behaviour

Windows

	CMD	PowerShell
Prompt	`C:\>`	`PS C:\>`
Output	Plain text	Objects (columns you can filter and sort)
Set a variable	`set AWS_DEFAULT_REGION=ap-south-1`	`$env:AWS_DEFAULT_REGION = "ap-south-1"`
Read a variable	`echo %AWS_DEFAULT_REGION%`	`$env:AWS_DEFAULT_REGION`
List all variables	`set`	`Get-ChildItem Env:`
Search text	`findstr /i "error"`	`Select-String -Pattern "error"` (alias `sls`)
Line continuation	`^` at line end	backtick `` ` `` at line end
Quotes	Only double quotes `"..."`	Single `'...'` = literal; double `"..."` = expands `$vars`
Escape character	`^`	backtick `` ` ``
Run as admin	Right-click, Run as administrator	Same
Scripts	`.bat`, `.cmd`	`.ps1` (may need `Set-ExecutionPolicy -Scope Process Bypass`)

Windows location

Topic	Linux PC	Windows + Docker Desktop
Container IPs (e.g. `10.20.1.10`)	Reachable from the host	**Not reachable** from Windows; use published ports
Reach an EC2 web app	`/etc/hosts` → container IP	Hosts file → `127.0.0.1`, plus a published port (SG forward `30000+`, see `docker ps`)
Hosts file	`/etc/hosts`	`C:\Windows\System32\drivers\etc\hosts`, edit as administrator
EC2 SSH	`ssh -p 2200 root@localhost`	Same; `ssh.exe` is built in
RDS	`localhost:7001`	Same, published by Floci
Docker socket in compose	`/var/run/docker.sock`	Same line still works with Docker Desktop (Linux engine)
Docker engine running?	`systemctl is-active docker`	Docker Desktop tray icon, or `docker info`
Memory and CPU	Whole PC	Limited by WSL2 / Docker Desktop settings

Topic	Linux PC	Windows + Docker Desktop
for containers		(`%UserProfile%\.wslconfig` or Docker Desktop → Resources)
Firewall	`ufw`	Windows Defender Firewall
File paths in compose	`./data`	`./data` works; avoid `C:\` paths with spaces; WSL paths are faster
Line endings	LF	Scripts edited on Windows may get CRLF; a `#!/bin/bash` user-data with CRLF fails inside Linux

```
# Docker and Floci
docker version
docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}
\t{{.Ports}}'
docker logs --tail 200 floci 2>&1 | Select-String -Pattern 'error|warn|
exception|fail|pull'
Get-PSDrive C | Select-Object Used, Free
Get-CimInstance Win32_OperatingSystem | Select-Object
@{n='FreeGB';e={[math]::Round($_.FreePhysicalMemory/1MB,1)}},
@{n='TotalGB';e={[math]::Round($_.TotalVisibleMemorySize/1MB,1)}}

# How Floci was started
docker inspect floci -f '{{json .HostConfig.PortBindings}}'
docker inspect floci -f '{{range .Config.Env}}{{println .}}{{end}}' |
Select-String floci
docker inspect floci -f '{{range .Mounts}}{{.Source}} -> {{.Destination}}
{{println}}{{end}}'

# API and CLI settings
curl.exe -s -o NUL -w '%{http_code}\n' http://localhost:4566
Get-ChildItem Env: | Where-Object Name -match 'AWS_|proxy'
aws configure list
```

```
New-Item -ItemType Directory -Force $HOME\triage | Out-Null; Set-Location
$HOME\triage
docker ps -a > containers-before.txt
docker inspect floci > floci-before.json
Copy-Item <their-folder>\docker-compose.yml compose-before.yml
-ErrorAction SilentlyContinue
```

```
docker logs floci 2>&1 | findstr /i "error warn fail pull", set |
findstr /i "AWS_ proxy", curl -s -o NUL -w "%{http_code}"
http://localhost:4566.
```

Docker host

```
docker info --format '{{.ServerVersion}} {{.OperatingSystem}}
mem={{.MemTotal}}'   # fails = Docker Desktop not running
wsl -l -v                                    # docker-desktop distro
should be Running
Get-PSDrive C | Select-Object Used, Free
docker system df
Get-Content $env:USERPROFILE\.wslconfig -ErrorAction SilentlyContinue   #
memory limit for WSL2
```

2. Floci container

```
docker ps -a --filter name=floci --format '{{.Names}}
{{.Status}}'
docker logs --tail 100 floci
docker inspect floci -f 'restarts={{.RestartCount}}
exit={{.State.ExitCode}} oom={{.State.OOMKilled}}'
```

3. Floci API and CLI

```
curl.exe -v http://localhost:4566 2>&1 | Select-Object -First 5
$env:AWS_ENDPOINT_URL = 'http://localhost:4566'
$env:AWS_ACCESS_KEY_ID = 'test'; $env:AWS_SECRET_ACCESS_KEY = 'test'
$env:AWS_DEFAULT_REGION = '<their region>'
$env:NO_PROXY = 'localhost,127.0.0.1'
aws sts get-caller-identity
```

`set AWS_ENDPOINT_URL=http://localhost:4566` and so on.

4. Resources

```
aws ec2 describe-instances --query 'Reservations[].Instances[].
[InstanceId,State.Name,PrivateIpAddress,ImageId]' --output table
aws rds describe-db-instances --query 'DBInstances[].
[DBInstanceIdentifier,Engine,DBInstanceStatus,Endpoint.Port]' --output
table
foreach ($r in 'us-east-1','us-west-2','eu-west-1','ap-south-1') { "$r "
+ (aws ec2 describe-instances --region $r --query 'length(Reservations)')
}
```

5. Spawned containers

```
docker ps -a --filter name=floci-aws --format 'table {{.Names}}
\t{{.Status}}\t{{.Ports}}'
docker logs --tail 50 <floci-aws-ec2-...|floci-aws-rds-...>
docker exec -it <floci-aws-ec2-...> bash      # Linux inside
```

6. Network (Windows side)

```
Test-NetConnection localhost -Port 4566      # TcpTestSucceeded : True
Test-NetConnection localhost -Port 7001
Get-NetTCPConnection -State Listen | Where-Object LocalPort -in
4566,7001,80,443 | Select-Object LocalAddress,LocalPort,OwningProcess
Get-Process -Id <OwningProcess>              # who owns that port
Resolve-DnsName shop.virto.local -ErrorAction SilentlyContinue; Get-
Content C:\Windows\System32\drivers\etc\hosts
```

```
Get-NetFirewallProfile | Select-Object Name, Enabled
docker exec <ec2> nc -zv -w 5 <host> <port>  # from inside the EC2 (Linux)
```

`netstat -ano | findstr :7001`, `tasklist /fi "PID eq <pid>"`, `nslookup <name>`, `type C:\Windows\System32\drivers\etc\hosts`.

ECS Fargate
aws ecs list-clusters
aws ecs list-services --cluster <c>
aws ecs describe-services --cluster <c> --services <s> \
  --query 'services[].
{desired:desiredCount,running:runningCount,pending:pendingCount,events:events[0:3].message}'
aws ecs list-tasks --cluster <c> --desired-status STOPPED # crashed tasks show here
aws ecs describe-tasks --cluster <c> --tasks <task-arn> \
  --query 'tasks[].{last:lastStatus,stopped:stoppedReason,containers:containers[].
{name:name,exit:exitCode,reason:reason}}'
aws logs tail <awslogs-group> --since 30m
docker ps -a --filter label=io.floci.owner

DynamoDB
aws dynamodb list-tables
aws dynamodb describe-table --table-name <t> --query 'Table.
{status:TableStatus,keys:KeySchema,attrs:AttributeDefinitions,gsi:GlobalSecondaryIndexes[].Inde
xName}'
aws dynamodb scan --table-name <t> --max-items 5
aws dynamodb get-item --table-name <t> --key '{"id":{"S":"123"}}'

VPC
aws ec2 describe-vpcs --output table
aws ec2 describe-subnets --query 'Subnets[].[SubnetId,VpcId,CidrBlock,AvailabilityZone]' --output table
aws ec2 describe-security-groups --query 'SecurityGroups[].[GroupId,GroupName,VpcId]' --output table
aws ec2 describe-security-groups --group-ids <sg> --query 'SecurityGroups[].IpPermissions' # rules
docker network ls ; docker network inspect <network>

Task	Linux bash	PowerShell	CMD
Set variable	`export X=1`	`$env:X = '1'`	`set X=1`
Read variable	`echo $X`	`$env:X`	`echo %X%`
Unset variable	`unset X`	`Remove-Item Env:X`	`set X=`
List variables	`env`	`Get-ChildItem Env:`	`set`
Search output	`| grep -i err`	`| Select-String -Pattern err`	`| findstr /i err`
Show file	`cat f`	`Get-Content f` (alias `cat`)	`type f`

Task	Linux bash	PowerShell	CMD
Last lines	`tail -50 f`	`Get-Content f -Tail 50`	(none; use PowerShell)
Follow a file	`tail -f f`	`Get-Content f -Wait -Tail 20`	(none)
List files	`ls -la`	`Get-ChildItem -Force` (alias `ls`, `dir`)	`dir /a`
Find a file	`find / -name x`	`Get-ChildItem C:\ -Recurse -Filter x -ErrorAction SilentlyContinue`	`dir C:\x /s`
Current folder	`pwd`	`Get-Location` (alias `pwd`)	`cd`
Make folder	`mkdir -p d`	`New-Item -ItemType Directory -Force d`	`mkdir d`
Copy	`cp a b`	`Copy-Item a b`	`copy a b`
Delete file	`rm f`	`Remove-Item f`	`del f`
HTTP status	`curl -s -o /dev/null -w '%{http_code}' url`	`curl.exe -s -o NUL -w '%{http_code}' url`	`curl -s -o NUL -w "%{http_code}" url`
Headers	`curl -I url`	`curl.exe -I url`	`curl -I url`
Port open?	`nc -zv host 7001`	`Test-NetConnection host -Port 7001`	`curl -v telnet://host:7001`
Who listens	`sudo ss -ltnp`	`Get-NetTCPConnection -State Listen`	`netstat -ano`
Process by PID	`ps -p 123`	`Get-Process -Id 123`	`tasklist /fi "PID eq 123"`
Kill process	`kill 123`	`Stop-Process -Id 123`	`taskkill /pid 123 /f`
Name lookup	`getent hosts n`	`Resolve-DnsName n`	`nslookup n`
Ping	`ping -c 2 ip`	`Test-Connection ip -Count 2`	`ping -n 2 ip`
Hosts file	`/etc/hosts`	`C:\Windows\System32\drivers\etc\hosts`	same
Edit hosts	`sudo nano /etc/hosts`	`notepad C:\Windows\System32\drivers\etc\hosts` (as admin)	same
Memory	`free -h`	`Get-CimInstance Win32_OperatingSystem`	`systeminfo | findstr Memory`
Disk	`df -h`	`Get-PSDrive C`	`wmic logicaldisk get size,freespace`

Task	Linux bash	PowerShell	CMD
Firewall	`sudo ufw status`	`Get-NetFirewallProfile`	`netsh advfirewall show allprofiles`
Run as admin	`sudo cmd`	Start PowerShell as administrator	Start CMD as administrator
Discard output	`> /dev/null`	`> $null` or `Out-Null`	`> NUL`
Line continuation	`\`	`` ` ``	`^`
Chain commands	`a && b`	`a; b` (5.1) or `a && b` (PowerShell 7)	`a && b`

Windows-specific problems

Symptom	Likely cause	Check	Fix
`error during connect ... docker_engine` / `The system cannot find the file specified`	Docker Desktop not running	Tray icon; `docker info`	Start Docker Desktop and wait for "Engine running"
Docker Desktop stuck starting	WSL2 broken or not enabled	`wsl -l -v`; `wsl --status`	`wsl --shutdown`, then restart Docker Desktop; `wsl --update`
`curl: A parameter cannot be found that matches 'w'`	`curl` is `Invoke-WebRequest` in PowerShell 5.1	`Get-Alias curl`	Type `curl.exe`
AWS `--query` fails with odd characters	PowerShell ate a backtick or `$` in double quotes	Look at the quotes	Use single quotes around `--query`
Docker `--format` prints the template literally in CMD	Single quotes do not quote in CMD	Shell used	Double quotes in CMD, or use PowerShell
Browser cannot reach `10.20.x.x` container IP	Docker Desktop VM network is not reachable from Windows	`Test-NetConnection 10.20.1.10 -Port 443` fails	Use a published port (`docker ps` shows SG forwards or add one) and `127.0.0.1`
Hosts file change ignored	Saved without admin rights, or as `hosts.txt`	`Get-Content` the file; check the name has no extension	Edit as admin; save as `hosts` with no extension; `ipconfig /flushdns`
User-data or script fails inside Linux with `\r` / `bad interpreter`	File has Windows CRLF line endings	`docker exec <c> cat -A /var/lib/user-data.sh` shows `^M`	Save as LF (VS Code: bottom bar CRLF → LF), or `dos2unix`
Containers OOM-killed though Windows has	WSL2 memory limit	`.wslconfig`; Docker	Raise `memory=` in `.wslconfig`, `wsl --`

Symptom	Likely cause	Check	Fix
free RAM		Desktop → Resources	`shutdown`, restart
Port already in use on Windows	Another app (IIS, Skype, Hyper-V reserved range)	`Get-NetTCPConnection -LocalPort <p>;` `netsh int ipv4 show excludedportrange protocol=tcp`	Stop the app or move the port outside reserved ranges
`.ps1` script will not run	Execution policy	`Get-ExecutionPolicy`	`Set-ExecutionPolicy -Scope Process Bypass` (this window only)
`aws` not found	AWS CLI not installed or not on PATH	`Get-Command aws`	Install the MSI, open a new window
Corporate proxy blocks image pulls	Proxy not set in Docker Desktop	Docker Desktop → Settings → Resources → Proxies	Set the proxy there; keep `NO_PROXY=localhost,127.0.0.1`
Copy-paste over RDP mangles commands	Smart quotes or lost line breaks	Look for “ ” or merged lines	Paste into Notepad first, or type short commands